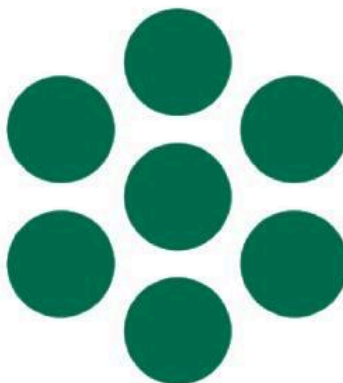


DEPARTAMENTO DE DESARROLLO

Soc Asesores



Documento:

Documento análisis de proyecto Biblioteca Virtual

Presenta:

Ingeniero Uriel Cisneros Torres

Fecha:

15 de Mayo del 2026, Casimiro Castillo Jalisco

Índice

1. Diagrama de Arquitectura.....	3
1.1 Capas del Sistema.....	3
1.2 Principio de Dependencias.....	3
1.3 Flujo de una Solicitud HTTP.....	4
2. Diagrama Entidad-Relación.....	5
2.1 Relaciones entre entidades.....	5
2.2 Definición de Tablas.....	5
3. Casos de Uso.....	9
CU-01 · Crear Usuario [USUARIOS].....	9
CU-02 · Inactivar Usuario [USUARIOS].....	9
CU-03 · Reactivar Usuario [USUARIOS].....	10
CU-04 · Otorgar Permisos a Asistente [USUARIOS].....	10
CU-05 · Iniciar Sesión (Login) [AUTENTICACIÓN].....	10
CU-06 · Crear Carpeta / Subcarpeta [CARPETAS].....	11
CU-07 · Subir Archivo [ARCHIVOS].....	11
CU-08 · Editar Metadata de Archivo [ARCHIVOS].....	12
CU-09 · Eliminar Archivo (Lógico) [ARCHIVOS].....	12
CU-10 · Visualizar y Descargar Archivo [ARCHIVOS].....	13

1. Diagrama de Arquitectura

El sistema implementa Clean Architecture con cuatro capas. Las dependencias fluyen estrictamente hacia el interior: API → Application → Domain ← Infrastructure.

1.1 Capas del Sistema

Proyecto	Capa	Responsabilidades principales
BibliotecaVirtual.API	Presentación	Controllers (Auth, Usuarios, Carpetas, Archivos, Logs), Middleware (ExceptionHandling, CurrentUser), Program.cs
BibliotecaVirtual.Application	Casos de Uso	Services (Auth, Usuario, Carpeta, Archivo, Log), DTOs, Validators (FluentValidation), Contracts/Interfaces
BibliotecaVirtual.Domain	Dominio (Núcleo)	Entities (Usuario, Carpeta, Archivo, LogAccion, HistorialUsuario), Enums, Interfaces de Repositorio, Excepciones
BibliotecaVirtual.Infrastructure	Infraestructura	AppDbContext (EF Core), Repositories, JwtTokenGenerator, BCryptPasswordHasher, AlmacenamientoArchivos, LimpiezaArchivosService

1.2 Principio de Dependencias

- La capa Domain no depende de ninguna otra capa ni librería externa.
- Application solo depende de Domain.
- Infrastructure implementa los contratos definidos en Domain y Application.
- API orquesta las capas de Application e Infrastructure, y expone los endpoints HTTP.

1.3 Flujo de una Solicitud HTTP

#	Componente	Acción
1	ExceptionHandlerMiddleware	Intercepta cualquier excepción no controlada y retorna una respuesta JSON estructurada.
2	JWT Auth Middleware	Valida el token Bearer; extrae Claims (correo, perfil, permisos).
3	CurrentUserMiddleware	Construye CurrentUserContext y lo registra en el DI container.
4	Controller	Recibe el request, valida el DTO con FluentValidation y llama al servicio de Application.
5	ApplicationService	Ejecuta la lógica de negocio, verifica permisos y llama al repositorio.
6	Repository (EF Core)	Traduce operaciones de dominio a consultas SQL contra SQLite.
7	Controller (respuesta)	Serializa el Result<T> a JSON y retorna el código HTTP correspondiente.

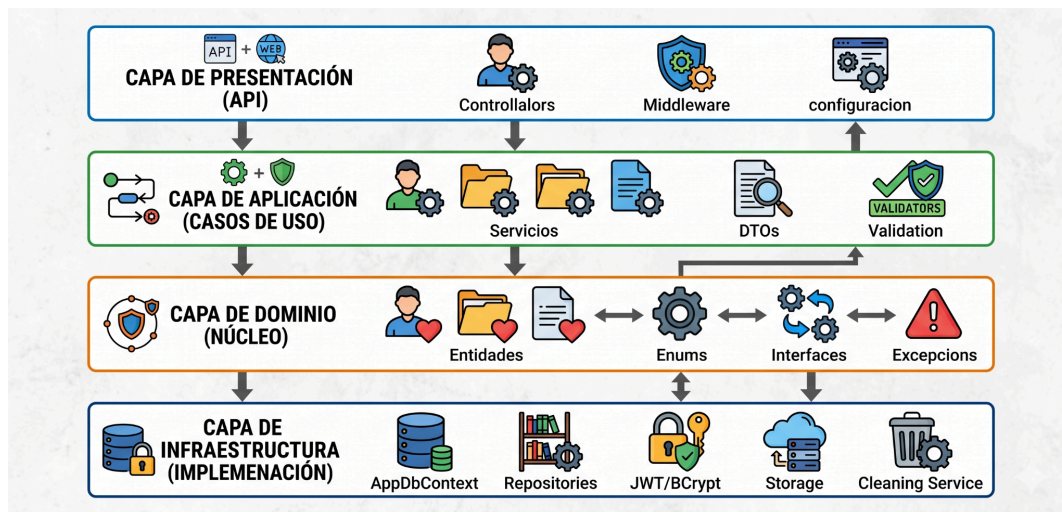


Figura 1.1 Diagrama visual de arquitectura.

2. Diagrama Entidad-Relación

La base de datos SQLite contiene 5 tablas. La clave primaria de Usuario es su correo electrónico. Las demás entidades heredan de BaseEntity (Id int autoincrement, FechaCreacion, Activo).

2.1 Relaciones entre entidades

- USUARIO 1 — * HISTORIAL_USUARIO (un usuario tiene múltiples registros de historial)
- CARPETA 1 — * CARPETA (auto-referencia: una carpeta tiene subcarpetas)
- CARPETA 1 — * ARCHIVO (una carpeta contiene múltiples archivos)

2.2 Definición de Tablas

USUARIO — Correo (PK)		
Campo	Tipo	Descripción
Correo	string	PK · Identificador único · No nulo
NombreCompleto	string	Nombre y apellidos
NombreUsuario	string	Username de pantalla
PasswordHash	string	Hash BCrypt de la contraseña
Perfil	int (enum)	AdministracionBiblioteca=1, GerenteUniversidad=2, AsistenteBiblioteca=3, Empleado=4
Activo	bool	true al crear; false al inactivar
PuedeGestionarArchivos	bool	Permiso extra para Asistentes
FechaCreacion	datetime	UTC · Se asigna al crear
FechaInactivacion	datetime?	Nullable · Se asigna al inactivar

HISTORIAL_USUARIO — Id (PK, autoincrement)		
Campo	Tipo	Descripción
Id	int	PK · Autoincrement
CorreoUsuario	string	FK → Usuario.Correo
Accion	int (enum)	TipoAccionLog
Detalle	string	Descripción de la acción
CorreoEjecutor	string	Quién realizó la acción
Fecha	datetime	UTC

CARPETA — Id (PK, autoincrement)		
Campo	Tipo	Descripción
Id	int	PK · Autoincrement (BaseEntity)
Nombre	string	Nombre de la carpeta
Descripcion	string?	Descripción opcional
CarpetaPadreId	int?	FK → Carpeta.Id · null si es raíz
EsRaiz	bool	true solo para carpetas de nivel raíz
CorreoCreador	string	FK → Usuario.Correo
FechaCreacion	datetime	UTC (BaseEntity)
Activo	bool	BaseEntity

ARCHIVO — Id (PK, autoincrement)		
Campo	Tipo	Descripción
Id	int	PK · Autoincrement (BaseEntity)
Nombre	string	Máximo 50 caracteres
Descripcion	string	Máximo 500 caracteres
RutaFisica	string	Ruta absoluta en disco
Extension	string	.pdf, .docx, .pptx, .xlsx, .mov, .avi, .mp3
TamanoBytes	long	Tamaño del archivo en bytes
Tipo	int (enum)	Documento=0, Video=1, Audio=2
CarpetaId	int	FK → Carpeta.Id
CorreoCreador	string	FK → Usuario.Correo
FechaCreacion	datetime	UTC (BaseEntity)
Activo	bool	false al eliminar lógicamente
FechaEliminacion Logica	datetime?	Inicia el contador de limpieza física

LOG_ACCION — Id (PK, autoincrement)		
Campo	Tipo	Descripción
Id	int	PK · Autoincrement
CorreoUsuario	string	Quién realizó la acción

Accion	int (enum)	TipoAccionLog
Entidad	string	"Usuario", "Carpeta" o "Archivo"
IdEntidad	string?	ID de la entidad afectada
Detalle	string	Descripción libre
Fecha	datetime	UTC

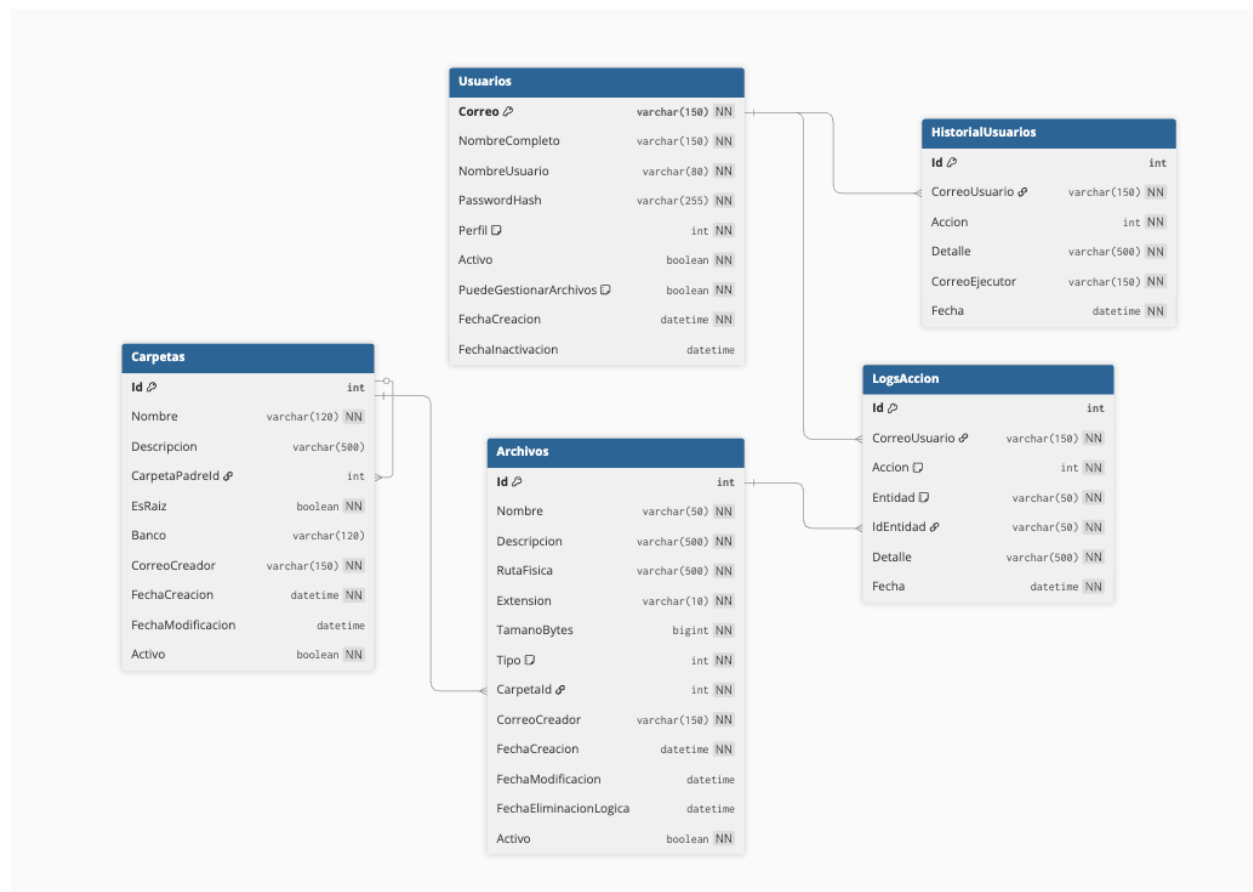


Figura 2.1 Diagrama entidad relacion

3. Casos de Uso

Los actores son: Administrador de Biblioteca, Gerente de Universidad, Asistente de Biblioteca (con/sin permiso) y Empleado.

CU-01 · Crear Usuario [USUARIOS]	
Actor Principal	Administrador de Biblioteca / Gerente de Universidad
Precondición	El actor está autenticado con perfil autorizado.
Flujo Principal	1. Ingresa nombre completo, usuario, correo y perfil. 2. El sistema valida que el correo no exista. 3. Se crea el usuario activo y se registra en HistorialUsuario.
Flujo Alternativo	Si el correo ya existe → alerta que determina que ya existe
Postcondición	Usuario creado con Activo = true, disponible para autenticarse.

CU-02 · Inactivar Usuario [USUARIOS]	
Actor Principal	Administrador de Biblioteca / Gerente de Universidad
Precondición	El usuario existe y está activo.
Flujo Principal	1. El actor selecciona un usuario activo. 2. El sistema marca Activo = false y FechaInactivacion = UTC now. 3. Se registra acción en HistorialUsuario con ejecutor y fecha.
Postcondición	El usuario no puede autenticarse. Sus datos se conservan.

CU-03 · Reactivar Usuario [USUARIOS]

Actor Principal	Administrador de Biblioteca / Gerente de Universidad
Precondición	El correo pertenece a un usuario inactivo.
Flujo Principal	1. Actor detecta usuario inactivo (alerta al intentar crear correo duplicado). 2. Confirma la reactivación. 3. El sistema marca Activo = true y registra la acción.
Postcondición	El usuario puede volver a autenticarse.

CU-04 · Otorgar Permisos a Asistente [USUARIOS]

Actor Principal	Administrador de Biblioteca / Gerente de Universidad
Precondición	El usuario objetivo tiene perfil AsistenteBiblioteca.
Flujo Principal	1. El actor selecciona un Asistente. 2. Activa el permiso PuedeGestionarArchivos = true. 3. El sistema registra la acción en LogAccion e HistorialUsuario.
Flujo Alternativo	Si el usuario no es Asistente → error de validación.
Postcondición	El Asistente puede crear carpetas y gestionar archivos.

CU-05 · Iniciar Sesión (Login) [AUTENTICACIÓN]

Actor Principal	Todos los usuarios
Precondición	El usuario existe y está activo.
Flujo Principal	1. Envía correo y contraseña a POST /api/auth/login. 2. El sistema valida credenciales con BCrypt. 3. Genera JWT con claims (correo, perfil, permisos). 4. Devuelve token con expiración de 60 minutos.
Flujo Alternativo	Credenciales inválidas o usuario inactivo → 401 Unauthorized.
Postcondición	Cliente obtiene JWT válido para usar en sucesivas solicitudes.

CU-06 · Crear Carpeta / Subcarpeta [CARPETAS]

Actor Principal	Admin, Gerente, Asistente (con permiso activo)
Precondición	Usuario autenticado con permisos de gestión.
Flujo Principal	1. Selecciona carpeta padre (o crea raíz si es Admin/Gerente). 2. Ingresa nombre y descripción. 3. El sistema valida unicidad y crea la carpeta con CorreoCreador.
Postcondición	Carpeta creada y disponible para navegación de todos los usuarios.

CU-07 · Subir Archivo [ARCHIVOS]

Actor Principal	Admin, Gerente, Asistente (con permiso activo)
------------------------	--

Precondición	La carpeta destino existe y está activa.
Flujo Principal	1. Selecciona carpeta destino. 2. Adjunta archivo (pdf, docx, pptx, xlsx, mov, avi, mp3). 3. Ingresa nombre (máx. 50) y descripción (máx. 500). 4. Sistema valida extensión y tamaño. 5. Guarda el archivo físico y persiste metadatos con fecha/creador.
Flujo Alternativo	Extensión no permitida o tamaño excedido → 400 Bad Request.
Postcondición	Archivo activo, visible y descargable por todos los usuarios.

CU-08 · Editar Metadata de Archivo [ARCHIVOS]

Actor Principal	Admin, Gerente, Asistente (con permiso activo)
Precondición	El archivo existe y está activo.
Flujo Principal	1. Envía PATCH /api/archivos/{id} con nuevo nombre y/o descripción. 2. El sistema valida longitudes. 3. Persiste cambios y registra acción en LogAccion.
Postcondición	Solo Nombre y Descripción actualizados. El archivo físico no cambia.

CU-09 · Eliminar Archivo (Lógico) [ARCHIVOS]

Actor Principal	Admin, Gerente, Asistente (con permiso activo)
Precondición	El archivo existe y está activo.
Flujo Principal	1. Envía DELETE /api/archivos/{id}. 2. Sistema marca Activo = false y FechaEliminacionLogica = UTC now. 3. Registra en LogAccion.

Postcondición	Archivo invisible para usuarios. LimpiezaArchivosService lo elimina físicamente tras 2 minutos (prueba) / 6 meses (producción).
----------------------	---

CU-10 · Visualizar y Descargar Archivo [ARCHIVOS]	
Actor Principal	Todos los usuarios autenticados
Precondición	El archivo existe y está activo.
Flujo Principal	1. Navega carpetas y localiza el archivo. 2. Obtiene metadata con GET /api/archivos/{id}. 3. Descarga el binario con GET /api/archivos/{id}/descargar.
Flujo Alternativo	Archivo eliminado lógicamente → 404 Not Found.
Postcondición	Usuario recibe el archivo físico con su ContentType correcto.